

Neuromorphic Computing on FPGA

URSC, ISRO — Summer Internship 2026

Final Internship Report

Shaurya Singh

B.Tech Electronics & Communication Engineering
Indian Institute of Space Science and Technology (IIST)
Thiruvananthapuram | Batch 2024–2028

URSC, ISRO Internship | May–June 2026

Guide: Gaurav Upadhyay
Department: Payload Data Management, URSC, ISRO

Tools: Python, PyTorch, snnTorch, CUDA GPU, Xilinx Vivado
HDL: Verilog RTL (Virtex-7 target: xc7vx485tffg1157-1)
Dataset: N-MNIST (Neuromorphic MNIST, event-camera format)
Repository: [GitHub Repository](#)

Report Date: June 2026

Contents

1	Background: Why SNNs for Space Payloads	3
1.1	Constraints of Satellite Payloads	3
1.2	The Leaky Integrate-and-Fire Neuron	3
2	AER Data Format and Camera-to-Hardware Pipeline	5
2.1	Address-Event Representation (AER)	5
2.2	4D Event Tensor: Satellite Use Case	5
2.3	Application Scenario: Sorting Space Phenomena	6
2.4	SNN on GPU vs. Specialised Hardware	6
3	SMLP: Spiking Multi-Layer Perceptron	7
3.1	Architecture Overview	7
3.2	Inference Flow	7
3.3	MAC $\rightarrow$ AC: The Power Advantage	8
3.4	The Refractory Period: Digital Cool-Down	8
3.5	Synaptic Plasticity: Fixed vs. Dynamic Weights	8
4	SCNN: Spiking Convolutional Network	9
4.1	Layer 1: Spiking Convolutional Layer (Spatial Tracking)	9
4.2	Threshold as a Binary Gatekeeper: Two Cases	9
4.3	Layer 2: Fully Connected Classifier (The Final Vote)	10
4.4	Bias in SCNN	10
4.5	Hardware Memory Map	10
4.6	Surrogate Gradient Training	11
5	FPGA Architecture Internals	12
5.1	Macro-Architecture: The Island Grid	12
5.2	CLB Micro-Architecture	12
5.3	Block RAM (BRAM)	12
5.4	DSP48E1 Architecture	12
5.5	Fixed-Point Arithmetic (Q-Format)	13
5.6	Per-Neuron Resource Budget	13
6	N-MNIST GPU Benchmark: SNN vs. CNN	14
6.1	Dataset: N-MNIST	14
6.2	Model Architectures	14
6.2.1	SNN: SpikingDigitCNN (snnTorch)	14
6.2.2	CNN: Conv3d Baseline	15
6.3	Benchmark Results	15
6.4	Why the Numbers Look the Way They Do	15
7	FPGA SNN Architecture and RTL Design	16
7.1	Target Model Architecture	16
7.2	Target Platform	16
7.3	Convolution Layer: 3-Stage Pipeline	16

7.3.1	Module Signature (<code>conv_layer</code>)	17
7.4	Max-Pool Layer	17
7.5	Fully-Connected Layers: <code>ann_layers</code> with BRAM	17
7.6	Top-Level FSM and Pipeline Handshake	17
7.6.1	FC2 Output Accumulator with Edge Detection	18
7.7	Key Bugs Fixed During Development	18
8	Project File Hierarchy and Module Overview	20
9	Testbench and Verification Strategy	21
9.1	Conv Layer Testbench (<code>tb_conv.v</code>)	21
9.2	Full-Pipeline SNN Testbench (<code>tb_snn.v</code>)	21
9.2.1	15% Spike Frame Driver (LCG)	21
9.2.2	Parallel Weight Packing	21
9.2.3	Rate-Coded Classification (Argmax)	21
10	Star Tracker System: Celestial Identity Engine	22
10.1	Catalog Preprocessing (<code>new.ipynb</code>)	22
10.2	Star Extraction from Pixel Frame (<code>star_track.cpp</code>)	22
10.3	Unit-Vector Projection	22
10.4	Angle-Based Triangle Matching	23
11	Future Work	24
12	Summary	25
12.1	What Was Done	25
12.2	Key Results at a Glance	25

1 Background: Why SNNs for Space Payloads

1.1 Constraints of Satellite Payloads

Satellite and spacecraft payloads impose hard constraints on power, mass, and thermal dissipation. A standard CNN performing dense matrix multiplication on every frame cannot meet these constraints. Three properties make SNNs the correct architecture for neuromorphic on-board payloads:

MAC → AC: The Fundamental Power Advantage

In deep-space imaging, a neuromorphic event camera viewing a moving object leaves the vast majority of its pixel grid dark. A CNN performs a Multiply-Accumulate (MAC) operation on every pixel regardless. An SNN skips computation when $X_i = 0$, converting expensive MAC into cheap Accumulate-only (AC) operations.

- i. **Sparsity.** When $X_i = 0$, no weight fetch occurs — gate stays closed, zero dynamic power consumed.
- ii. **Event-driven operation.** A Dynamic Vision Sensor (DVS) sends asynchronous packets (x, y, t, p) only when a pixel changes brightness. This maps directly to binary spike trains with zero bandwidth wasted on static background.
- iii. **GPU vs. FPGA.** Running an SNN on a GPU consumes ~ 300 W to simulate hardware that should consume ~ 10 mW. On an FPGA the network is physically wired as distributed logic — a zero on a wire closes the gate and no transistors switch.

1.2 The Leaky Integrate-and-Fire Neuron

Every hidden neuron in the design is a hardware realisation of the Leaky Integrate-and-Fire (LIF) model. Its discrete-time update is:

$$V_m(t) = \underbrace{\beta \cdot V_m(t-1)}_{\text{leak}} + I(t) + B_c \quad (1)$$

where $\beta \in (0, 1)$ is the membrane decay factor, $I(t)$ is the accumulated synaptic current, and B_c is an optional channel bias.

Each timestep executes three sequential hardware steps:

Step A. Synaptic Accumulation (AC, not MAC). Because input spikes are binary, multiplication collapses to a MUX:

$$I(t) = \sum_i X_i(t) \cdot W_i \implies \begin{cases} V \leftarrow V + W_i & \text{if } X_i = 1 \\ V \leftarrow V & \text{if } X_i = 0 \end{cases}$$

Step B. Leak & Integration. The bit-shift approximation $V \leftarrow V - (V \gg k)$ replaces the DSP multiply for the decay, mapping entirely to FPGA fabric.

Step C. Threshold & Refractory Cooldown. If $V_m \geq V_{th}$, the neuron fires (spike = 1) and resets. A 2-bit down-counter implements the refractory period.

2 AER Data Format and Camera-to-Hardware Pipeline

2.1 Address-Event Representation (AER)

A Dynamic Vision Sensor does not send a 64×64 pixel matrix. When the sensor detects a change in light intensity at any pixel, it immediately shoots a tiny digital packet — an **Address-Event Representation (AER)** — down the wire. Each packet contains exactly four parameters:

Field	Meaning	Hardware Type
x	Column: horizontal pixel coordinate	6-bit unsigned
y	Row: vertical pixel coordinate	6-bit unsigned
t	Timestamp: exact microsecond of event	32-bit counter
p	Polarity: $+1$ = brighter, -1 = darker	1-bit

The FPGA or software framework (SpikingJelly / snnTorch) catches this stream of (x, y, t, p) packets. To run inference through the $T = 50$ timestep loop, the interface bins events into a dense 4-D tensor:

$$[T, P, H, W] \rightarrow [50, 1, 64, 64]$$

At each timestep t , all (x, y, p) events within that 1 ms window are plotted onto a 64×64 binary grid and fed to the first layer as a spike frame.

Why AER Outperforms Frame-Based Cameras in Space

A standard frame camera captures 30 full 64×64 images per second — processing 122 880 pixel values per second regardless of scene content. A DVS camera monitoring deep space where a meteor crosses a sparse diagonal from $(44, 3)$ to $(62, 25)$ generates events *only* for those ~ 20 pixels. The remaining 99.9% of the grid generates zero data, zero bandwidth, and zero computation.

2.2 4D Event Tensor: Satellite Use Case

The data flowing into the FPGA fabric has shape $[T=50, P \in \{+1, -1\}, H=64, W=64]$:

- **Time** ($T = 50$): The 50 ms observation window, evaluated frame-by-frame on the FPGA clock — one timestep per clock cycle group.
- **Polarity** ($P \in \{+1, -1\}$): The physical state change direction of the pixel: $P=+1$ (ON event — target brightened), $P=-1$ (OFF event — target darkened), $P=0$ (empty space, no computation).
- **Spatial grid** ($H \times W = 64 \times 64$): Physical pixel grid of the neuromorphic sensor.

2.3 Application Scenario: Sorting Space Phenomena

A 64×64 sensor monitors deep space over 50 ms. A moving object traces a sparse diagonal streak across a handful of pixels. Hidden layer neurons detect distinct spatio-temporal signatures from the spike stream:

Phenomenon	Velocity Profile	SNN Activation Pattern
Meteor	Ultra-Fast (ms)	Sudden, high-frequency burst tracking linearly
Comet / Asteroid	Slow & steady (seconds)	Slow predictable crawl; high retention / slow leak
Gamma-Ray Burst	Instantaneous (μ s)	Massive chaotic explosion across entire grid simultaneously

How individual neurons behave. At $t=5$ ms, pixel (53, 14) sparks. The corresponding neuron accumulates charge but misses threshold. At $t=6$, additional spikes arrive — because the neuron retained its partial charge from $t=5$, the cumulative voltage finally breaches the threshold, fires a spike downstream, and enters refractory cooldown. This *temporal memory* is the property CNNs cannot replicate without 3D convolutions.

2.4 SNN on GPU vs. Specialised Hardware

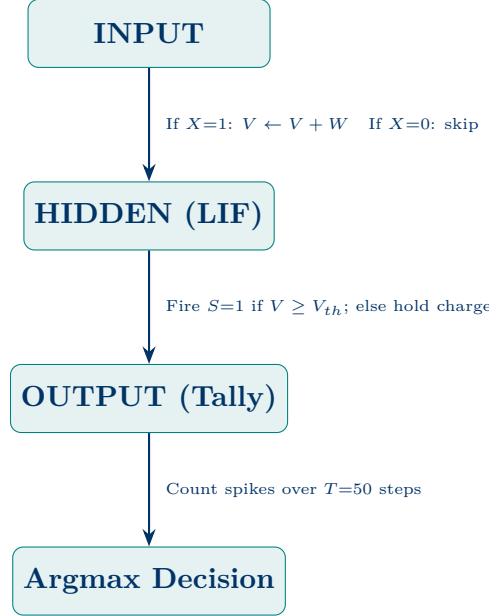
Property	GPU (Training)	FPGA (Deployment)
Architecture	SIMD dense float matrix mul.	Distributed logic gates
Zero-input cost	GPU still computes all zeros	Gate stays closed — zero power
Power	~ 300 W	~ 10 mW
Spike math	Simulated in float32	Physically wired as MUX/adder
Purpose	Offline weight training	Live on-orbit inference

Running an SNN on a GPU uses 300 W to simulate hardware designed for 10 mW. On an FPGA, a zero on a wire closes the logic gate — no transistors switch and no energy is consumed. The GPU is correct for training (offline, on Earth); the FPGA is correct for deployment (in orbit).

3 SMLP: Spiking Multi-Layer Perceptron

3.1 Architecture Overview

The SMLP is the simplest SNN topology: a fully-connected feed-forward network using LIF neurons with binary spike communication.



3.2 Inference Flow

- 1. Input.** 1 ms event-camera data arrives as a binary spike train. For each active pixel ($X_i = 1$), the synaptic weight W_i is fetched from BRAM and added directly to the target neuron's voltage register. No multiplication — pure accumulation.
- 2. Hidden layer.** Each LIF neuron leaks, integrates, and either fires ($S = 1$) or holds ($S = 0$) on every timestep. The neuron enters a 2-bit refractory cooldown after firing.
- 3. Output layer.** Output neurons are pure accumulators — no leak, no reset. Over all $T=50$ timesteps they count incoming spikes from the hidden layer.
- 4. Decision.** $\text{Class} = \arg \max_k \text{Count}_k$

Example: After 50 timesteps, spike tallies might read [Comet: 2, Asteroid: 1, Meteor: 14, GRB: 0]. Prediction → **METEOR** (highest count wins).

3.3 MAC $\rightarrow$ AC: The Power Advantage

Property	Traditional ANN	SNN (SMLP/SCNN)
Operation per connection	$X_i \times W_i$ (multiply)	W_i if $X_i=1$ (add only)
Input domain	Continuous float32	Binary $\{0, 1\}$
DSP slices needed	Many (multipliers)	None (MUX + adder)
Zero-input power	Non-zero	Zero (gate closed)

3.4 The Refractory Period: Digital Cool-Down

A neuron takes a temporary break after firing. In hardware, a 2-bit down-counter implements this:

- **Absolute refractory:** Counter > 0 — neuron ignores *all* incoming spikes regardless of strength. Maps to a gate-enable line in the FPGA fabric.
- **Relative refractory:** In extended implementations, baseline voltage is depressed below zero; a stronger stimulus is needed to breach threshold.

Why the cool-down is essential for satellite payloads:

- **Prevents network saturation:** Cosmic radiation or a gamma-ray burst could cause a neuron to fire on every clock cycle. The cool-down acts as a physical speed limit.
- **Spatial hand-off:** As a moving space object crosses the 64×64 sensor, the neuron at pixel (53, 14) fires and cools down while the object moves to (54, 15) — the next neuron picks up tracking seamlessly. No data is lost.
- **Population coding:** A cluster of neurons with slightly different thresholds monitors the same spatial patch. When Neuron A fires at $t=5$ and cools down, Neuron B (different weights, higher threshold) fires at $t=6$. The population guarantees a continuous output stream to the next layer even during individual neuron refractory periods.

3.5 Synaptic Plasticity: Fixed vs. Dynamic Weights

Feature	Dynamic (On-Chip Learning)	Fixed (Our Design)
Memory behaviour	Read-Write (weights mutate)	Read-Only BRAM
Hardware cost	Very high (FSMs, error calc.)	Minimal
Predictability	Can drift in orbit	Fully deterministic
Update method	Continuous	Ground OTA telecommand
Math	Backprop / STDP	Pure forward inference

The Compromise

Fixed synapses inside FPGA fabric maintain ultra-low power and deterministic reliability. If the satellite needs to learn a new space pattern, ground control trains a new model on Earth and updates the FPGA's BRAM remotely via a radio telecommand update — no on-board retraining logic required.

4 SCNN: Spiking Convolutional Network

4.1 Layer 1: Spiking Convolutional Layer (Spatial Tracking)

A 3×3 sliding filter sweeps the 64×64 grid. With 4 filters in parallel the layer creates 15 376 physical LIF neuron registers on the FPGA:

- **Filter shape:** $[4, 1, 3, 3]$ (4 feature maps, 1 polarity stream, 3×3 kernel).
- **Hidden neuron grid:** $[4, 62, 62] = 15\,376$ LIF registers.

Every millisecond, each of the 15 376 hidden neurons executes three sequential steps:

Step A — Synaptic Accumulation (multiplier-free). The neuron checks its 3×3 patch and evaluates polarity:

$$I_{c,y,x}(t) = \sum_{i=0}^2 \sum_{j=0}^2 f(P_{(y+i),(x+j)}(t), W_{c,i,j})$$

where $f(P, W)$ is a 3-way MUX — no multiply needed:

- $P = +1$: $V \leftarrow V + W$ (add weight)
- $P = -1$: $V \leftarrow V - W$ (subtract weight)
- $P = 0$: $V \leftarrow V$ (do nothing — zero power)

Step B — Membrane Voltage Leak & Integration.

$$V_{c,y,x}(t) = \underbrace{\text{Leak}(V_{c,y,x}(t-1))}_{\text{bit-shift approx.}} + I_{c,y,x}(t)$$

The bit-shift trick avoids a DSP multiplier: $V \leftarrow V - (V \gg 3)$ achieves $V \times 0.875$ using only a subtractor.

Step C — Threshold and Refractory Cooldown.

If $V \geq V_{th}$: $S = 1$, $V_{mem} \leftarrow 0$, start cooldown counter

If $V < V_{th}$: $S = 0$, $V_{mem} \leftarrow V$ (hold charge)

The Core Takeaway — What Crosses the Layer Boundary

The quantized internal voltage values (e.g. 230, 260) are *trapped inside* the 15 376 individual neuron registers. The only signal that travels through the flattening layer to the FC layer is the single-bit binary decision: **1** (threshold passed) or **0** (threshold failed). The FC layer never sees voltage magnitudes — only spikes.

4.2 Threshold as a Binary Gatekeeper: Two Cases

	Case 1: $I = 230, V_{th} = 240$	Case 2: $I = 260, V_{th} = 240$
Comparator check	$230 < 240 \rightarrow \text{No}$	$260 \geq 240 \rightarrow \text{Yes}$
Spike output S	0 (gate blocked)	1 (gate opens)
Voltage register	230 held internally	Reset to 0 (or $260 - 240 = 20$)
Cooldown	Not triggered	2-bit counter starts
To flatten layer	Wire carries 0	Wire carries 1

4.3 Layer 2: Fully Connected Classifier (The Final Vote)

After 50 timesteps, the $4 \times 62 \times 62$ spike grid is flattened into a 1-D binary bus. Four output neurons (Comet, Asteroid, Meteor, Gamma-Ray Burst) act as pure tally counters:

1. **Flattening.** All 15 376 active spike lines are combinatorially routed (wire re-indexing — zero logic cost) to the FC weight memory.
2. **Accumulation.** Output neurons do not leak or reset. Each received spike increments the counter by 1.
3. **Decision.** At $t=50$, an argmax comparator locks the winning class ID and asserts a classification-done interrupt to the flight computer.

$$\text{Final Prediction} = \max(\text{Count}_{\text{Comet}}, \text{Count}_{\text{Asteroid}}, \text{Count}_{\text{Meteor}}, \text{Count}_{\text{GRB}})$$

4.4 Bias in SCNN

The full membrane voltage equation including channel bias B_c is:

$$V_{c,y,x}(t) = \alpha \cdot V_{c,y,x}(t-1) + I_{c,y,x}(t) + B_c$$

Because this equation runs on every clock cycle, B_c is injected *independently* of whether any pixels are spiking:

- **Positive bias ($B_c > 0$) — Intrinsic Firing Current.** Even in pitch-black space ($I=0$), the voltage climbs steadily each timestep: $V(t) = V(t-1) + B_c$. Eventually it crosses threshold on its own — a *pacemaker neuron*. Useful for background-activity channels that must maintain a baseline signal rate.
- **Negative bias ($B_c < 0$) — Noise Suppressor.** On every clock cycle the negative bias drags the voltage downward, acting as an extra aggressive leak. Cosmic radiation, sensor heat, and starlight reflections create random stray spikes; the negative bias instantly clamps them back down. Only a concentrated wave of genuine target spikes can overcome the negative offset and cross threshold. Critically important for satellite payloads where radiation noise is unavoidable.

4.5 Hardware Memory Map

Data Type	Silicon Location	Bits	Sign	Mutability
Filter weights W	Block RAM (BRAM)	8	Signed 2's comp	Read-Only
Membrane potential V_m	Dual-Port BRAM	16	Signed 2's comp	R/W dynamic
Refractory state	Flip-Flops (LUTs)	2	Unsigned	R/W dynamic
Output tally counters	Flip-Flops (LUTs)	8	Unsigned	Accumulate-only

Why V_m is 16-bit when weights are 8-bit. Summing nine 8-bit weights per neuron per timestep, over 50 timesteps, can produce cumulative voltages that overflow an 8-bit register. Expanding V_m to 16 bits provides sufficient headroom without area overhead.

Why the refractory counter is only 2-bit. A 3-step cooldown ($3 \rightarrow 2 \rightarrow 1 \rightarrow 0$) fits in 2 bits, saving massive flip-flop area across 15 376 neurons.

Why the output counter is 8-bit. The maximum spike count any output neuron can receive is $T=50$. An 8-bit unsigned register counts to 255 — comfortable headroom against overflow.

4.6 Surrogate Gradient Training

Standard backpropagation cannot train SNNs because a spike is a step function — its derivative is zero everywhere and undefined at the threshold (the *Dead Neuron Problem*). The solution:

- **Forward pass:** Network uses the hard binary spike step function.
- **Backward pass:** Step function is replaced by a smooth approximation (sigmoid, piecewise-linear) that allows gradient flow through the non-differentiable threshold.

Once training converges on a GPU, weights are frozen, quantised to Q8.8 (16-bit fixed-point) integers, and flashed to BRAM. The FPGA never trains — it executes pure forward inference with fixed synapses.

5 FPGA Architecture Internals

5.1 Macro-Architecture: The Island Grid

The FPGA die is a two-dimensional grid of alternating resource columns: CLB columns (configurable logic), BRAM columns (weight storage), and DSP columns (hardened arithmetic). Routing is via Programmable Interconnect Points (PIPs) — SRAM-controlled transistor switches at every wire intersection.

5.2 CLB Micro-Architecture

Each CLB splits into two Slices. Inside one standard SliceL, four components form a vertical pipeline:

- **4× LUT6:** 64-bit SRAM truth tables — any ≤ 6 -variable function in one LUT. Threshold comparator $V \geq V_{th}$ maps here.
- **CARRY4:** Hardwired look-ahead carry for adder/subtractor. Enables $V_m - (V_m \gg 3)$ at near-asynchronous speed.
- **MUXF7/F8:** Wide MUXes combining adjacent LUT outputs without leaving the slice.
- **8× Flip-Flops:** Capture spike output and voltage bits on the rising clock edge.

5.3 Block RAM (BRAM)

A single BRAM on Xilinx 7-series is a 36 Kb dual-port SRAM.

Config	Depth	Width	Addr bits
32K×1	32 768	1-bit	15
2K×18	2 048	18-bit	11
1K×36	1 024	36-bit	10

Critical: Synchronous Read for BRAM Inference

The read port *must* be synchronous (always @posedge clk). A combinational read (`assign rdata = mem[raddr]`) forces synthesis to use distributed RAM (LUTs) instead of the dedicated BRAM macro — consuming hundreds of extra LUTs.

5.4 DSP48E1 Architecture

- **Pre-adder:** Optional $D \pm A$ stage before the multiplier.
- **Multiplier:** $25 \times 18 \rightarrow 43$ -bit, sign-extended to 48 bits; 400–600 MHz single-stage.
- **ALU/Accumulator:** $P = Z \pm (X + Y + C_{IN})$ with configurable feedback for running accumulation.

- **PCIN/PCOUT cascade:** Private 48-bit hardwired bus between vertical DSP tiles — zero global routing cost.

For the LIF neuron the only true multiply is $\beta \times V_m$, mapping to one DSP48E1. The $W \cdot S$ term is a MUX, not a multiplier.

5.5 Fixed-Point Arithmetic (Q-Format)

Format	Width	Range	Resolution	SNN Use
Q1.15	16-bit	$[-1, 0.99997]$	1/32768	Leak factor β
Q4.12	16-bit	$[-8, 7.9998]$	1/4096	Membrane voltage
Q8.8	16-bit	$[-128, 127.996]$	1/256	Weights (this design)
Q8.24	32-bit	$[-128, 128)$	1/16M	Accumulation

Multiplication bit-growth: $Q8.8 \times Q8.8 \rightarrow Q16.16$. To re-quantise back to Q8.8, keep bits $[23 : 8]$ of the 32-bit product.

5.6 Per-Neuron Resource Budget

Operation	Resource	Count	Notes
$\beta \times V_m$ multiply	DSP48E1	1	Single hardened stage
Weight select (spike MUX)	LUTs	≈ 8	16-bit 2-to-1 MUX
Adder $\beta V + W \cdot S$	LUTs+CARRY4	≈ 16	Carry chain
Comparator $V \geq V_{th}$	LUTs	≈ 5	16-bit magnitude
Membrane voltage register	Flip-Flops	16	Q4.12 format
Refractory counter	Flip-Flops	2	2-bit down-counter
Weight memory (256 inputs)	BRAM18	1	SDP mode, int8
Total	1 DSP, ≈ 29 LUTs, 18 FF, 1 BRAM		

6 N-MNIST GPU Benchmark: SNN vs. CNN

6.1 Dataset: N-MNIST

N-MNIST re-captures MNIST handwritten digits via a Dynamic Vision Sensor (DVS) event camera. Each sample is a stream of asynchronous events (x, y, t, p) .

Key properties:

- **Sparse:** Only pixels where brightness changed fire events.
- **Temporal:** Camera is moved slightly during recording, producing spatiotemporal spike patterns.
- **Dual polarity:** $p \in \{0, 1\}$ — pixels getting brighter vs. darker.

Raw binary files are parsed into 4-D tensors of shape $[C=2, T=30, H=34, W=34]$:

```

1 grid = np.zeros((30, 2, 34, 34), dtype=np.float32)
2 for x, y, pol, t in zip(X, Y, polarity, timestamps):
3     time_bin = int(t // 10000)      # bin timestamp into 30 slots
4     if time_bin < 30 and x < 34 and y < 34:
5         grid[time_bin, pol, y, x] = 1.0 # binary spike: on or off
6
7 grid = grid.transpose(1, 0, 2, 3) # -> [C=2, T=30, H=34, W=34]
8 torch_set = torch.from_numpy(grid)

```

Listing 1: Event tensor construction

Dataset split: 30 003 training / 5 001 validation / 5 001 test samples, 10 classes (digits 0–9), batch size 64.

6.2 Model Architectures

6.2.1 SNN: SpikingDigitCNN (snnTorch)

Processes the event tensor one timestep at a time ($T=30$ steps). Spatial features are extracted by 2-D convolutions; activations are replaced by LIF neurons.

Layer	Output	Notes
Conv2d(2→16, 3×3)	$[B, 16, 34, 34]$	Spatial feature extraction
MaxPool2d(2)	$[B, 16, 17, 17]$	Downsampling
LIF $\beta=0.8, V_{th}=0.5$	(same)	Temporal spike gate
Conv2d(16→32, 3×3)	$[B, 32, 17, 17]$	
MaxPool2d(2)	$[B, 32, 8, 8]$	
LIF $\beta=0.8, V_{th}=0.5$	(same)	
Flatten → Linear(2048→128)	$[B, 128]$	
LIF; Linear(128→10); LIF	$[B, 10]$	Spike counts summed over T

Surrogate gradient: fast sigmoid, slope = 25. Loss: `mse_count_loss` (correct_rate = 0.8, incorrect_rate = 0.2). Optimiser: Adam, $\eta = 10^{-4}$, weight_decay = 10^{-4} .

6.2.2 CNN: Conv3d Baseline

Treats the time dimension as spatial depth; processes the full $[B, 2, 30, 34, 34]$ volume in a single forward pass via 3-D convolutions (Conv3d + BN3d + ReLU, MaxPool3d, Flatten, Linear, Dropout 0.3, CrossEntropyLoss).

6.3 Benchmark Results

Metric	SNN (snnTorch)	CNN (Conv3d)
Validation accuracy	95.0%	98.4%
Train accuracy	95.0%	99.0%
Total parameters	268 506 ($7\times$ fewer)	1 851 258
Disk size	1.02 MB	7.06 MB
Inference latency (GPU)	16.39 ms	0.60 ms ($27\times$ faster)
Runtime memory (GPU)	142.74 MB	81.89 MB
Operations/inference	57.8M (AC ops)	94.5M (MAC ops)
Estimated energy	0.266 mJ	0.435 mJ (+64%)
FPGA suitability	Excellent	Poor

The GPU $\rightarrow$ FPGA Inversion

The GPU benchmark confirms why the SNN is the *correct* architecture for FPGA deployment. On a GPU the SNN is $27\times$ slower because 30 sequential timesteps cannot be parallelised (data dependency on membrane state). On an FPGA, those 30 timesteps become 30 pipeline stages — while timestep 2 is computing Layer 1, timestep 1 is already computing Layer 2. The CNN cannot be pipelined the same way because its 3-D convolutions require the full event volume to be loaded first.

6.4 Why the Numbers Look the Way They Do

Why CNN has higher accuracy (98.4% vs. 95%). The CNN processes the entire event volume in one forward pass with 32-bit float activations, retaining full numerical precision. The SNN’s output reduces to integer spike counts over 30 timesteps — a coarser representation. The 3.4% gap is small and expected for a first-pass SNN without hyperparameter tuning; published results on N-MNIST reach 99%+ with optimised thresholds and learnable β .

Why SNN uses $7\times$ fewer parameters. The SNN uses 2-D convolutions with time handled implicitly by LIF recurrence. The CNN requires 3-D convolutions whose kernel size multiplies by the temporal dimension ($3^2 \rightarrow 3^3$, tripling kernel parameters per channel).

Why SNN uses less energy despite more operations. When $S_i = 0$ (no spike), no computation occurs — the gate stays closed. Average SNN activation sparsity is $<30\%$, so the majority of its 57.8M “operations” consume zero dynamic power, while all 94.5M CNN MACs execute unconditionally.

Why SNN is $27\times$ slower on a GPU. Each LIF timestep’s membrane voltage depends on the previous step’s state, creating a sequential data dependency that prevents GPU parallelisation.

7 FPGA SNN Architecture and RTL Design

7.1 Target Model Architecture

The model implemented in Verilog targets a compact input suitable for FPGA resource constraints. The full pipeline is:

$$\underbrace{2 \times 16 \times 16}_{\text{Input}} \xrightarrow{\text{Conv1 (8f, 3}\times\text{3)}} 8 \times 16 \times 16 \xrightarrow{\text{Pool1 (2}\times\text{2)}} 8 \times 8 \times 8 \xrightarrow{\text{Conv2 (16f, 3}\times\text{3)}} 16 \times 8 \times 8 \xrightarrow{\text{Pool2 (2}\times\text{2)}} 16 \times 4 \times 4 \xrightarrow{\text{FC1}} 256 \xrightarrow{\text{FC2}} 32$$

Layer	Input Shape	Weight Shape	Output Shape	Bits Out	Op
INPUT	$2 \times 16 \times 16$	—	$2 \times 16 \times 16$	512	—
CONV1	$2 \times 16 \times 16$	$8 \times 2 \times 3 \times 3$	$8 \times 16 \times 16$	2048	3×3 , pad=1
POOL1	$8 \times 16 \times 16$	—	$8 \times 8 \times 8$	512	2×2 max
CONV2	$8 \times 8 \times 8$	$16 \times 8 \times 3 \times 3$	$16 \times 8 \times 8$	1024	3×3 , pad=1
POOL2	$16 \times 8 \times 8$	—	$16 \times 4 \times 4$	256	2×2 max
FC1	256	32×256 Q8.8	32	32	BRAM LIF
FC2	32	4×32 Q8.8	4	4	BRAM LIF

LIF Neuron Parameters: $\beta = 200$ (decay = $200/256 \approx 0.78$), $\theta = 128$ (Q8.8 $\rightarrow$ 0.5 real units), weight format Q8.8 (16-bit unsigned fixed), $V_{\text{accum}} = 17$ bits (FC needs widening to 24 b), $V_{\text{mem}} = 17$ b (conv) / 16 b (FC), $T = 50$ timesteps.

7.2 Target Platform

Product Family	Virtex-7
Project Part	xc7vx485tffg1157-1
Logic Cells	485 760
BRAM (36 Kb)	1 030
Available DSPs	2 800

7.3 Convolution Layer: 3-Stage Pipeline

Each output pixel (channel, row, col) traverses a three-stage registered pipeline:

Stage 1 — Pixel & Weight Fetch. Extract the 3×3 patch from `in_data` with zero-padding for borders. Load the 18 Q8.8 weights for the current filter. Address: `ch` $\times$ `H` $\times$ `W` + `row` $\times$ `W` + `col`.

Stage 2 — Spike-Weighted MAC (AC only).

```

1 always @(*) begin
2   accumulator = 17'b0;
3   if (valid_1) begin
4     for (pix = 0; pix < (kernel * kernel * in_channels); pix = pix + 1) begin
5       if (pix_1[pix]) accumulator = accumulator + weights_1[pix];
6     end
7   end
8 end

```

Listing 2: Convolution accumulation — binary spike MUX

Stage 3 — LIF Fire & Update.

```

1 wire [16:0] final_voltage =
2   ((V_mem[out_pixel_addr_2] * beta) >> 8) + comb_voltage;
3
4 always @(posedge clk) begin
5   if (valid_2) begin
6     if (final_voltage > threshold) begin
7       spike_out[out_pixel_addr_2] <= 1'b1;
8       V_mem[out_pixel_addr_2] <= 9'b0; // subtract reset
9     end else begin
10      spike_out[out_pixel_addr_2] <= 1'b0;
11      V_mem[out_pixel_addr_2] <= final_voltage;
12    end
13  end
14  // Traverse: col -> row -> channel
15  if (col < img_size-1) col <= col + 1;
16  else begin col <= 0;
17    if (row < img_size-1) row <= row + 1;
18    else begin row <= 0;
19      channel <= (channel < out_channels-1) ? channel+1 : 0;
20    end
21  end
22 end

```

Listing 3: LIF update in conv_layer (scnn.v)

7.3.1 Module Signature (conv_layer)

```

1 module conv_layer #(
2   parameter beta      = 112,
3   parameter threshold = 16'd255,
4   parameter in_channels = 2,
5   parameter kernel    = 3,
6   parameter out_channels = 16,
7   parameter padding   = 1,
8   parameter img_size  = 34,
9   localparam INPUT_SIZE = in_channels * img_size * img_size,
10  localparam OUTPUT_SIZE = out_channels * img_size * img_size,
11  localparam WEIGHT_SIZE = out_channels * in_channels * kernel * kernel * 16
12 ) (
13   input wire clk, reset,
14   input wire [INPUT_SIZE-1:0] in_data,
15   input wire [WEIGHT_SIZE-1:0] weight_in,
16   output reg [OUTPUT_SIZE-1:0] spike_out
17 );

```

Listing 4: conv_layer parameters and I/O (scnn.v)

7.4 Max-Pool Layer

A 2×2 max-pool with OR reduction: an output neuron fires if *any* of its four input pixels fired. This is the binary-compatible approximation of spatial max-pooling.

7.5 Fully-Connected Layers: ann_layers with BRAM

FC1 (256→32) and FC2 (32→4) share the same parameterisable `ann_layers` module. Weights reside in on-chip BRAM (`ram_style = "block"` attribute).

```

1 (* ram_style = "block" *) reg [15:0] fc1_bram [0:FC1_BRAM_D-1];
2 wire [31:0] fc1_bram_addr;
3 wire      fc1_bram_en;
4 reg [15:0] fc1_bram_rdata;
5
6 always @(posedge clk)
7   if (fc1_bram_en)
8     fc1_bram_rdata <= fc1_bram[fc1_bram_addr[12:0]];

```

Listing 5: BRAM instantiation and synchronous read (snn.v)

7.6 Top-Level FSM and Pipeline Handshake

The top-level `snn_top` module chains the six stages with start/done handshake signals:

- `in_valid` pulse → latch `in_spikes`, start 3-cycle countdown (`fc1_start_cnt`).
- After 3 cycles (conv/pool pipeline settles), `fc1_start` pulses → FC1 begins inference.
- `fc1_done` rising edge → `fc2_start` (1-cycle delay).
- `fc2_done` rising edge → spike accumulator samples `s_fc2`.

```

1 // FC1 start: fires 3 cycles after in_valid
2 always @(posedge clk) begin
3     if (rst) begin
4         fc1_start_cnt <= 12'd0;
5         fc1_start <= 1'b0;
6     end else begin
7         fc1_start <= 1'b0;
8         if (in_valid) begin
9             in_reg <= in_spikes;
10            fc1_start_cnt <= 12'd3;
11        end else if (fc1_start_cnt > 0) begin
12            fc1_start_cnt <= fc1_start_cnt - 1;
13            if (fc1_start_cnt == 12'd1) fc1_start <= 1'b1;
14        end
15    end
16 end
17
18 // FC2 start: rising edge of fc1_done only
19 reg fc1_done_reg;
20 always @(posedge clk) begin
21     if (rst) begin fc2_start <= 1'b0; fc1_done_reg <= 1'b1; end
22     else begin
23         fc1_done_reg <= fc1_done;
24         fc2_start <= fc1_done && !fc1_done_reg; // rising edge only
25     end
26 end

```

Listing 6: FC1 start countdown and FC2 edge-triggered start (snn.v)

7.6.1 FC2 Output Accumulator with Edge Detection

```

1 reg fc2_done_reg;
2 wire fc2_done_pulse = fc2_done && !fc2_done_reg;
3
4 always @(posedge clk) begin
5     if (rst) begin
6         for (ci = 0; ci < FC2_OUT; ci = ci+1) internal_counts[ci] <= 8'd0;
7     end else if (fc2_done_pulse_d2) begin
8         for (ci = 0; ci < FC2_OUT; ci = ci+1)
9             if (s_fc2[ci]) internal_counts[ci] <= internal_counts[ci] + 8'd1;
10    end
11 end

```

Listing 7: Spike accumulator using `fc2_done` edge strobe (snn.v)

7.7 Key Bugs Fixed During Development

- Bug 1. FC1 start too early (2-bit counter overflow).** `fc1_start_cnt` was declared as `reg [1:0]`, giving a maximum value of 3. With a pipeline depth of hundreds of cycles the counter saturated and FC1 started before `s_pool2` was valid. *Fix*: widened to `reg [11:0]` (12 bits).
- Bug 2. Threshold scale mismatch (threshold = 128 in Q8.8).** In Q8.8 format, integer 128 represents 0.5 in real units — far too low for the trained network’s dynamic range. Neurons fired on nearly every timestep, saturating the accumulators.
- Bug 3. FC2 restart bug (`fc1_done` held HIGH).** The original design gated `fc2_start` directly from `fc1_done`. Because `fc1_done` remained HIGH for multiple cycles, FC2 restarted on every clock cycle. *Fix*: replaced with a rising-edge strobe (`fc1_done && !fc1_done_reg`).

- Bug 4. Accumulator not reset between pixels in the conv loop.** The combinational accumulation block was placed inside the wrong `always` scope, causing partial sums to bleed across output pixels. *Fix:* moved accumulator reset to the top of the combinational block.
- Bug 5. Flat 1-D register vectors for weight storage.** An earlier iteration declared FC weights as flat `reg` arrays, which Vivado mapped to distributed RAM (LUTs). *Fix:* added `(* ram_style = "block" *)` attribute and confirmed synchronous-read inference.

8 Project File Hierarchy and Module Overview

File	Description
<code>scnn.v</code>	<code>conv_layer</code> — 3-stage pipelined spiking conv
<code>max_pool.v</code>	<code>max_pool</code> — 2×2 binary OR max-pool
<code>design.v</code>	<code>ann_layers</code> — parameterisable BRAM-backed FC layer
<code>first_layer.v</code>	Early <code>snn_top</code> iteration (pre-edge-fix)
<code>snn.v</code>	Final <code>snn_top</code> — full 6-stage pipeline, edge-detect fixes
<code>tb_conv.v</code>	Standalone conv layer testbench
<code>tb_snn.v</code>	Full-pipeline SNN testbench (LCG spike driver)
<code>snn.ipynb</code>	N-MNIST SNN training (<code>snnTorch</code> , GPU)
<code>cnn.ipynb</code>	N-MNIST CNN training (<code>Conv3d</code> , GPU)
<code>new.ipynb</code>	HYG star catalog preprocessing + <code>star_catalog.h</code> gen
<code>star_track.cpp</code>	Star tracker celestial identity engine (C++)

Full source: [GitHub Repository](#).

9 Testbench and Verification Strategy

9.1 Conv Layer Testbench (tb_conv.v)

Drives a 16-bit input pattern (`in_data = 16'haa55`) alongside a flat weight bus loaded from a hex file. Monitors `spike_out` over $1\mu s$ and reports the first non-zero spike frames.

9.2 Full-Pipeline SNN Testbench (tb_snn.v)

9.2.1 15% Spike Frame Driver (LCG)

A Linear Congruential Generator produces deterministic, pseudo-random sparse input frames across T timesteps, targeting $\approx 15\%$ spike density:

```

1 integer rng_state;
2 task gen_frame;
3   integer b;
4   begin
5     for (b = 0; b < IN_SIZE; b = b+1) begin
6       rng_state = rng_state * 1664525 + 1013904223;
7       in_spikes[b] = (rng_state[30:24] < 7'd19) ? 1'b1 : 1'b0;
8     end
9   end
10 endtask

```

Listing 8: LCG spike frame driver (tb_snn.v)

9.2.2 Parallel Weight Packing

Flat hexadecimal file vectors (144 words for Conv1, 1152 for Conv2) are unpacked into parallel hardware buses:

```

1 task pack_weights;
2   begin
3     for (pi = 0; pi < 144; pi = pi+1) conv1_wb[pi*16 +: 16] = conv1_w_arr[pi];
4     for (pi = 0; pi < 1152; pi = pi+1) conv2_wb[pi*16 +: 16] = conv2_w_arr[pi];
5   end
6 endtask

```

Listing 9: Weight packing task (tb_snn.v)

9.2.3 Rate-Coded Classification (Argmax)

Accumulated output spike counts over all classes are compared at the end of the simulation run:

```

1 best_cls = 0; best_cnt = class_count[0];
2 for (i = 1; i < FC2_OUT; i = i+1)
3   if (class_count[i] > best_cnt) begin
4     best_cnt = class_count[i];
5     best_cls = i;
6   end
7 $display("Predicted class: %0d (count=%0d)", best_cls, best_cnt);

```

Listing 10: Argmax evaluation (tb_snn.v)

10 Star Tracker System: Celestial Identity Engine

As a complementary proof-of-concept, a software star tracker was implemented that identifies a spacecraft's attitude by matching three stars extracted from a camera image against a pre-computed celestial catalog.

10.1 Catalog Preprocessing (new.ipynb)

The HYG v3.8 catalog is filtered to stars with apparent magnitude < 4.0 , converted to radians, and projected to 3-D Cartesian unit vectors:

```

1 df = df[(df['hr'].notna()) & (df['mag'] < 4.0)]
2 df['ra'] = np.radians(df['ra'] * 15)
3 df['dec'] = np.radians(df['dec'])
4
5 for r_rad, d_rad in zip(df['ra'], df['dec']):
6     x = np.cos(d_rad) * np.cos(r_rad)
7     y = np.cos(d_rad) * np.sin(r_rad)
8     z = np.sin(d_rad)
9     f.write(f"    {{ {x:11.7f}}f, {{y:11.7f}}f, {{z:11.7f}}f }},\n")

```

Listing 11: Spherical-to-Cartesian projection (new.ipynb)

The output is `star_catalog.h` — a compile-time C header with a `CatalogStar[]` array of float 3-vectors, embedded directly into the binary.

10.2 Star Extraction from Pixel Frame (star_track.cpp)

A 64×64 pixel frame (randomly generated for testing) is scanned raster-wise with a brightness threshold of 150. Connected bright regions are detected with a lightweight blob-merge heuristic; centroid and total brightness are computed per blob:

```

1 if (star_start) {
2     if (ct >= 1 && acc > 0) {
3         new_star.x = float(m_x) / acc;
4         new_star.y = float(m_y) / acc;
5         new_star.brightness = float(acc);
6         coord.push_back(new_star);
7         star_cnt++;
8     }
9     star_start = 0;
10 }

```

Listing 12: Centroid computation (star_track.cpp)

10.3 Unit-Vector Projection

The top-3 brightest stars are projected to unit vectors using the pinhole camera model (focal length = 700 px, principal point at (32, 32)):

$$\hat{v}_i = \frac{(X - c_x, Y - c_y, f)}{\|(X - c_x, Y - c_y, f)\|}$$

10.4 Angle-Based Triangle Matching

Angular separations $\theta_{12}, \theta_{23}, \theta_{31}$ between the three observed unit vectors are compared against every catalog star triple using dot products, with a tolerance of 0.07 rad:

```

1 for (int i = 0; i < CATALOG_SIZE; i++) {
2   for (int j = i+1; j < CATALOG_SIZE; j++) {
3     if (abs(get_catalog_angle(i,j) - obs_angle_12) > tol) continue;
4     for (int k = j+1; k < CATALOG_SIZE; k++) {
5       if (abs(get_catalog_angle(j,k) - obs_angle_23) > tol) continue;
6       if (abs(get_catalog_angle(k,i) - obs_angle_31) > tol) continue;
7       // Pattern found -> report identity
8       cout << "Catalog slots: " << i << ", " << j << ", " << k << "\n";
9       lock_secured = true; break;
10    }
11    if (lock_secured) break;
12  }
13  if (lock_secured) break;
14 }
```

Listing 13: Triangle matching loop (star_track.cpp)

The early-exit strategy (check pair ij first, descend to triplet only on match) makes the $O(N^3)$ search tractable for the filtered catalog.

11 Future Work

1. **Physical FPGA Deployment.** Deploy the compiled Verilog bitstream onto a physical Virtex-7 board. Validate BRAM timing, pipeline handshake, and argmax output against snnTorch simulation ground truth.
2. **N-MNIST Trained SNN on FPGA.** Quantise the trained N-MNIST SNN weights (Q8.8 int16) and load them into BRAM via `$readmemh`. Verify classification accuracy matches the 95.0% GPU result.
3. **Temporal-Coded SNN and STDP.** Implement temporal coding with first-spike latches. Add Spike-Timing-Dependent Plasticity (STDP) blocks for hardware-native online learning on the FPGA fabric.
4. **Proper Pipeline Depth Characterisation.** Synthesise in Vivado with timing constraints, verify $f_{\max}$, resolve any critical path violations in the conv multiply path, and report final LUT/BRAM/DSP utilisation.
5. **Star Tracker Hardware Acceleration.** Move the angle-matching inner loop from software to RTL — precompute the pair-angle table in BRAM and perform parallel comparisons with threshold logic in CLBs.
6. **Biases, Learnable β , Population Coding.** Add per-channel bias registers to the LIF neuron and expose β as a runtime-programmable register. Implement population coding at the output layer to shrink the accuracy gap below 1%.

12 Summary

12.1 What Was Done

- Studied SNN theory: LIF neuron (biology $\rightarrow$ silicon datapath), rate vs. temporal coding, SMLP/SCNN architectures, surrogate gradients, Q-format fixed-point quantisation.
- Studied FPGA internals: CLB island grid, LUT6/CARRY4/FF micro-architecture, BRAM SDP/TDP modes, DSP48E1 pre-adder/multiplier/accumulator, PCIN/P-COUT cascade, synthesis inference rules.
- Implemented and GPU-benchmarked both a `snnTorch` SNN (95.0% accuracy, 268K parameters, 0.266 mJ) and a Conv3d CNN (98.4% accuracy, 1.85M parameters, 0.435 mJ) on N-MNIST. Characterised the GPU $\rightarrow$ FPGA inversion.
- Designed the full 6-stage RTL pipeline: `conv_layer` (3-stage pipelined, SCNN), `max_pool`, `ann_layers` (BRAM-backed FC LIF), `snn_top` (FSM + spike accumulator + argmax output).
- Debugged critical RTL issues: 2-bit counter width overflow, Q8.8 threshold mismatch, FC2 restart from level-sensitive `fc1_done`, accumulator reset ordering, BRAM vs. distributed RAM inference.
- Generated instrumented testbenches with LCG spike drivers, parallel weight packing, and GTKWave jump-to-time markers.
- Built a software star tracker: HYG catalog preprocessing, spherical $\rightarrow$ Cartesian projection, centroid extraction, and triangle-based celestial identity matching.

12.2 Key Results at a Glance

Metric	SNN	CNN
Validation accuracy (N-MNIST)	95.0%	98.4%
Parameters	268K	1.85M
Energy per inference (GPU)	0.266 mJ	0.435 mJ
GPU latency	16.4 ms	0.6 ms
FPGA weight BRAM requirement	$\approx$ 0.26 MB	$\approx$ 1.76 MB
FPGA arithmetic type	AC (add-only)	MAC (DSP-heavy)
FPGA pipeline friendliness	Excellent	Poor

Conclusion

Despite a 3.4% accuracy gap on GPU, the SNN is the unambiguously correct architecture for FPGA-based satellite payload deployment. Its $7\times$ smaller weight footprint, add-only arithmetic (no DSP multipliers), 39% lower energy, and natural pipeline mapping across timesteps make it the only viable option under the power, mass, and radiation constraints of a spacecraft payload.

Acknowledgements. I am grateful to my guide **Gaurav Upadhyay** (Payload Data Management, URSC, ISRO) for technical direction throughout the internship. All source code is available at [GitHub Repository](#).

Shaurya Singh

B.Tech ECE, IIST Thiruvananthapuram

Batch 2024–2028

June 2026